

DocProcessor

Превратите документацию в верифицируемую карту функций для автоматизации QA.

Краткое описание

DocProcessor — это автономный, полностью изолированный модуль Go, который загружает проектную документацию, формирует структурированные карты функций и отслеживает покрытие верификации. Он предназначен для работы с агентами LLM для интеллектуального извлечения функций, но также включает эвристический парсер для офлайн-использования.

Описание

Проектно-независимый модуль Go для обработки документации и извлечения карт функций. Он преобразует документацию в структурированные карты и отслеживает, какие функции верифицированы — с помощью агентов LLM для интеллектуального извлечения или эвристик в офлайн-режиме — обеспечивая автоматизацию QA гарантией соответствия реальности без обмана.

Подробное описание

Каждая команда разработчиков живёт с одной и той же медленной ложью: документация обещает одни функции, тесты покрывают что-то похожее, и никто не может с уверенностью сказать, описывают ли они один и тот же продукт. DocProcessor создан, чтобы сделать этот разрыв видимым и измеримым. На основе проектной документации он формирует структурированную карту функций — перечисленную, машиночитаемую модель всего, что продукт заявляет как свои возможности, — и отслеживает покрытие верификации по отношению к ней. Теперь вопрос «Доказано ли, что эта задокументированная функция действительно работает?» перестаёт быть предметом споров в кулуарах и

превращается в запрос с однозначным ответом. Модуль намеренно работает в двух режимах: использует агентов LLM для интеллектуального, семантического извлечения функций, когда они доступны, и переключается на эвристический парсер для полностью офлайн-работы. Это гарантирует, что он никогда не зависит жёстко от наличия модели и одинаково функционирует как в изолированной CI-задаче, так и на ноутбуке разработчика в режиме полёта.

С архитектурной точки зрения это автономный, проектно-независимый, полностью изолированный модуль Go (CONST-051(B)): он не содержит специфичных для проекта значений и подключается потребителями как равноправный подмодуль в кодовой базе. Это позволяет любому проекту внедрить его без наследования чужих допущений. Кроме того, модуль придерживается тех же стандартов, которые навязывает другим: его собственные заявления подчиняются антиобманному соглашению (CONST-035) и правилам полного покрытия автоматизацией (CONST-048). Это означает, что каждая возможность, заявленная в README, проверяется автоматизированным тестом или скриптом Challenge, подтверждающим реальное, пригодное для конечного пользователя поведение, а не просто завершающимся без ошибок. Пользовательские строки проходят через шов переводчика i18n (CONST-046). Смысл всего этого — замкнутый цикл: DocProcessor — это входная часть цикла QA, который завершает HelixQA. Он извлекает карту функций из документации, а HelixQA доказывает каждую функцию из карты с помощью собранных данных о работе системы. В результате документация, тесты и фактическое поведение продукта вынуждены сходиться, а не тихо расходиться от релиза к релизу.

Зачем мы его создали

Документация и тесты расходятся: в документации обещают функции, которые не подтверждаются тестами, а QA не может легко определить, что означает «полное покрытие». DocProcessor превращает документацию в машиночитаемую карту функций, чтобы покрытие верификации можно было измерять относительно того, что было действительно заявлено.

Почему это меняет правила игры

Оно превращает самый размытый вопрос в процессе поставки ПО — *«соответствует ли то, что мы выпустили, тому, что мы обещали выпустить?»* — в нечто автоматизируемое и поддающееся непрерывной проверке. При этом решение не

привязано к жёсткой зависимости AI: извлечение LLM происходит при наличии модели, а в её отсутствие — с помощью эвристик. Таким образом, одна и та же гарантия работает в любых условиях — от офлайн-запуска до полностью автоматизированного конвейера.

Что нового

- Извлечение карты соответствий между документацией и функциональностью с отслеживанием покрытия верификации.
- Двойное извлечение: с помощью агентов LLM или эвристик/офлайн-режима.
- Независимость от проекта и отвязка от конфигурации (CONST-051(B)).
- Антиблефовая самопроверка: утверждения в README подкреплены тестами/испытаниями (CONST-035/048).

Проблемы и решения

- **Работа без модели:** решено за счёт эвристического экстрактора, позволяющего модулю функционировать офлайн.
- **Синхронизация документации и реальности:** решено с помощью структурированных карт функциональности и отслеживания покрытия верификации, интегрированных в цикл контроля качества.
- **Повторное использование:** решено за счёт строгой отвязки и потребления подмодулей из единой кодовой базы.
- **Доверие к собственным заявлениям:** решено с помощью антиблефовых тестов/испытаний для каждой заявленной возможности.

Технологический стек (зачем и как)

- **Go (1.25+)** — ядро модуля; лицензия Apache-2.0.
- **Агенты LLM** — интеллектуальное семантическое извлечение функциональности (опционально).
- **Эвристический парсер** — резервное извлечение функциональности в офлайн-режиме.
- **Переводчик i18n (pkg/i18n)** — локализованные строки по CONST-046.
- **Фреймворк испытаний** — антиблефовая верификация собственных заявлений модуля.